

Building a React Native Time Tracking App: A Complete Beginner's Guide

Introduction

Welcome! This tutorial is designed for people with **zero programming experience**. We'll explain everything step-by-step, including what programming languages are, what different words mean, and why we use them.

By the end of this tutorial, you'll have built a complete mobile app that tracks employee time and job sites. More importantly, you'll understand how programming works.

Part 0: Programming Basics

What is Programming?

Programming is giving instructions to a computer. Think of it like writing a recipe for a chef:

Recipe (Instructions for a chef):

1. Get 2 eggs
2. Crack them into a bowl
3. Beat them with a fork
4. Heat a pan
5. Pour eggs into pan
6. Wait 2 minutes
7. Flip eggs
8. Wait 1 minute
9. Put on plate

Programming is similar, but we're giving instructions to a computer:

Program (Instructions for a computer):

1. Ask the user for their name
2. Store the name in memory
3. Display "Hello, [name]!"

What is a Programming Language?

A programming language is a way to communicate with computers. Just like humans speak English, Spanish, French, etc., computers have different programming languages:

- **JavaScript** — Used for web pages and mobile apps
- **Python** — Used for data analysis and automation
- **Java** — Used for large applications
- **C++** — Used for games and high-performance software

In this tutorial, we'll use **JavaScript** and **TypeScript**.

JavaScript vs TypeScript

JavaScript is a programming language that runs in web browsers and mobile apps. It's been around since 1995 and is the most popular language for web development.

TypeScript is a newer language built on top of JavaScript. It adds extra features that help catch mistakes before they happen. Think of it like:

- **JavaScript**: Writing a letter without spell-check
- **TypeScript**: Writing a letter with spell-check that catches errors before you send it

For this tutorial, we'll use TypeScript because it helps us avoid bugs.

Part 1: JavaScript Fundamentals

Before we can understand React Native, we need to understand JavaScript. Let's start with the basics.

Variables: Storing Information

A **variable** is a container that holds information. Think of it like a labeled box:

```
Box labeled "name" contains: "John"  
Box labeled "age" contains: 25  
Box labeled "isWorking" contains: true
```

In JavaScript, we create variables using the `let` keyword:

```
let name = "John";  
let age = 25;  
let isWorking = true;
```

Breaking this down:

- `let` — A keyword that means “create a new variable”
- `name` — The name of the variable (what we call the box)
- `=` — Means “assign” or “put this value into the variable”
- `"John"` — The value (what goes in the box)
- `;` — Ends the statement (like a period at the end of a sentence)

Why use variables? Because we need to remember information. If we ask a user for their name, we need to store it somewhere so we can use it later.

Data Types: Different Kinds of Information

Different types of information need different storage. JavaScript has several data types:

String (Text)

A **string** is text. We write it with quotes:

```
let name = "John Smith";  
let address = "123 Main Street";  
let message = "Hello, World!";
```

Why quotes? Because without them, JavaScript thinks `John` is a variable name, not text.

Number

A **number** is, well, a number:

```
let age = 25;  
let salary = 50000.50;  
let temperature = -5;
```

Numbers can be whole numbers (integers) or have decimals.

Boolean (True/False)

A **boolean** is either `true` or `false`. It's used for yes/no questions:

```
let isWorking = true;  
let hasLicense = false;  
let isAdmin = true;
```

Null (Nothing)

`null` means “nothing” or “no value”:

```
let clockOutTime = null; // Employee hasn't clocked out yet
```

We use `null` when something doesn't have a value yet.

Operators: Doing Things With Variables

Operators are symbols that do actions. Here are the common ones:

Arithmetic Operators (Math)

```
let a = 10;
let b = 3;

let sum = a + b;      // 13 (addition)
let difference = a - b; // 7 (subtraction)
let product = a * b;   // 30 (multiplication)
let quotient = a / b;  // 3.33... (division)
let remainder = a % b; // 1 (modulo - leftover after division)
```

Comparison Operators (Comparing)

These compare two values and return `true` or `false` :

```
let age = 25;

age == 25;    // true (equals)
age !== 25;   // false (not equals)
age > 20;     // true (greater than)
age < 30;     // true (less than)
age ≥ 25;     // true (greater than or equal)
age ≤ 25;     // true (less than or equal)
```

Logical Operators (And/Or/Not)

```
let age = 25;
let hasLicense = true;

age > 18 && hasLicense; // true (AND - both must be true)
age > 18 || hasLicense; // true (OR - at least one must be true)
!hasLicense;           // false (NOT - flips true/false)
```

Functions: Reusable Instructions

A **function** is a set of instructions that we can reuse. Think of it like a recipe:

```
function greetUser(name) {  
  console.log("Hello, " + name + "!");  
}  
  
greetUser("John");    // Prints: Hello, John!  
greetUser("Jane");    // Prints: Hello, Jane!
```

Breaking this down:

- `function` — Keyword that means “I’m defining a function”
- `greetUser` — Name of the function
- `(name)` — Parameter (input) - the function receives a value called `name`
- `{ ... }` — The body (the instructions inside the function)
- `console.log()` — Prints text to the console
- `+` — Concatenates (joins) strings together

Why use functions? Because you often need to do the same thing multiple times. Instead of writing the same code over and over, you write it once in a function and call it whenever you need it.

Objects: Grouping Related Information

An **object** is a container that holds related information. Think of it like a person’s profile:

```
let employee = {  
  name: "John Smith",  
  age: 25,  
  jobTitle: "Landscape",  
  isWorking: true  
};
```

Breaking this down:

- `let employee = { ... }` — Create a variable called `employee` that contains an object

- `name: "John Smith"` — A property (key-value pair). `name` is the key, `"John Smith"` is the value
- `,` — Separates properties

To access properties, use a dot:

```
console.log(employee.name);    // Prints: John Smith
console.log(employee.age);     // Prints: 25
console.log(employee.jobTitle); // Prints: Landscaper
```

You can also change properties:

```
employee.age = 26; // Change age to 26
employee.name = "John Doe"; // Change name
```

Arrays: Lists of Things

An **array** is a list of items. Think of it like a shopping list:

```
let employees = ["John", "Jane", "Bob", "Alice"];
```

To access items in an array, use the index (position). **Important: Counting starts at 0, not 1!**

```
employees[0]; // "John" (first item)
employees[1]; // "Jane" (second item)
employees[2]; // "Bob" (third item)
employees[3]; // "Alice" (fourth item)
```

To add items to an array:

```
employees.push("Charlie"); // Adds "Charlie" to the end
// Now: ["John", "Jane", "Bob", "Alice", "Charlie"]
```

To get the number of items:

```
employees.length; // 5
```

To remove an item:

```
employees.pop(); // Removes the last item
// Now: ["John", "Jane", "Bob", "Alice"]
```

Loops: Repeating Instructions

A **loop** repeats instructions multiple times. There are different types:

For Loop

```
for (let i = 0; i < 3; i = i + 1) {
  console.log("Hello " + i);
}

// Output:
// Hello 0
// Hello 1
// Hello 2
```

Breaking this down:

- `let i = 0` — Start with `i` equal to 0
- `i < 3` — Keep looping while `i` is less than 3
- `i = i + 1` — After each loop, add 1 to `i`

For-Each Loop

This is simpler when you want to loop through an array:

```
let employees = ["John", "Jane", "Bob"];

employees.forEach(function(name) {
  console.log(name);
});

// Output:
// John
// Jane
// Bob
```

This says: “For each name in the employees array, print the name.”

Conditional Statements: Making Decisions

Conditional statements let your program make decisions:

```
let age = 25;

if (age ≥ 18) {
  console.log("You are an adult");
} else {
  console.log("You are a child");
}

// Output: You are an adult
```

Breaking this down:

- `if (condition)` — If the condition is true, run the code inside `{ }`
- `else` — If the condition is false, run this code instead

You can also have multiple conditions:

```
let age = 25;

if (age < 13) {
  console.log("Child");
} else if (age < 18) {
  console.log("Teenager");
} else {
  console.log("Adult");
}

// Output: Adult
```

Part 2: TypeScript: Adding Safety to JavaScript

TypeScript is JavaScript with extra safety features. The main feature is **types**.

What are Types?

A **type** tells the computer what kind of data a variable should hold. Think of it like a label on a box:

```
Box labeled "age" should only contain: numbers
Box labeled "name" should only contain: text
Box labeled "isWorking" should only contain: true/false
```

In TypeScript, we specify types like this:

```
let name: string = "John";    // This variable must be text
let age: number = 25;         // This variable must be a number
let isWorking: boolean = true; // This variable must be true or false
```

The `: string`, `: number`, and `: boolean` are **type annotations**. They tell TypeScript what type each variable should be.

Why is this useful? If you accidentally try to put the wrong type in a variable, TypeScript catches the mistake:

```
let age: number = 25;
age = "twenty-five"; // ERROR! You can't put text in a number variable
```

TypeScript will show an error before you even run the program, saving you from bugs.

Interfaces: Describing Objects

An **interface** is a blueprint that describes what an object should look like. Think of it like a form:

```
Employee Form:
- Name (must be text)
- Age (must be a number)
- Job Title (must be text)
- Is Working (must be true/false)
```

In TypeScript, we write this as:

```
interface Employee {
  name: string;
  age: number;
  jobTitle: string;
  isWorking: boolean;
}
```

Now we can use this interface to create objects:

```
let employee: Employee = {  
  name: "John Smith",  
  age: 25,  
  jobTitle: "Landscape",  
  isWorking: true  
};
```

If we try to create an employee without all the required fields, TypeScript will show an error:

```
let employee: Employee = {  
  name: "John Smith",  
  age: 25  
  // ERROR! Missing jobTitle and isWorking  
};
```

Why use interfaces? They ensure that all your data follows the same structure. If you have 100 employees, they all have the same fields in the same order.

Union Types: Multiple Possible Types

Sometimes a variable can be one of several types. We use the `|` (pipe) symbol:

```
let value: string | number;  
  
value = "hello"; // OK - it's a string  
value = 42;      // OK - it's a number  
value = true;    // ERROR - it's not a string or number
```

We use this when something might be one of a few types:

```
let clockOutTime: number | null;

clockOutTime = 1717707600000; // OK - it's a number
clockOutTime = null;          // OK - it's null
clockOutTime = "3:00 PM";     // ERROR - it's not a number or null
```

This is useful because `clockOutTime` might be a timestamp (number) or nothing (null) if the employee hasn't clocked out yet.

Part 3: React Fundamentals

React is a JavaScript library (a collection of code) that makes building user interfaces easier. It's used for web apps and mobile apps.

What is a User Interface (UI)?

A user interface is what you see on the screen. It includes:

- Text
- Buttons
- Input fields
- Images
- Lists
- Everything the user interacts with

Components: Building Blocks

A **component** is a reusable piece of UI. Think of it like a LEGO brick:

```
LEGO brick (component): A button
You can use this brick many times to build different things
```

In React, we create components as functions:

```
function Button() {  
  return <button>Click Me</button>;  
}
```

Breaking this down:

- `function Button()` — Define a function called `Button`
- `return` — Return what should appear on screen
- `<button>Click Me</button>` — This looks like HTML (web markup language), but it's actually JSX (more on this next)

JSX: HTML-like Syntax in JavaScript

JSX is a way to write HTML-like code inside JavaScript. It makes code easier to read:

```
// Without JSX (hard to read):  
React.createElement('button', null, 'Click Me')  
  
// With JSX (easy to read):  
<button>Click Me</button>
```

JSX looks like HTML, but it's actually JavaScript. When the code runs, JSX is converted to JavaScript.

Common JSX elements:

<code><View></code>	// Like a <code><div></code> in HTML - a container
<code><Text></code>	// Like a <code><p></code> in HTML - text
<code><TextInput></code>	// Like an <code><input></code> in HTML - input field
<code><Pressable></code>	// Like a <code><button></code> in HTML - clickable element
<code><ScrollView></code>	// A scrollable container
<code><FlatList></code>	// A list that efficiently renders many items

Props: Passing Data to Components

Props are how you pass data to components. Think of them like function parameters:

```
function Greeting(props) {  
  return <Text>Hello, {props.name}!</Text>;  
}  
  
// Usage:  
<Greeting name="John" /> // Displays: Hello, John!  
<Greeting name="Jane" /> // Displays: Hello, Jane!
```

Breaking this down:

- `props` — An object that contains all the data passed to the component
- `props.name` — Access the `name` property from `props`
- `name="John"` — Pass a prop called `name` with value `"John"`
- `{props.name}` — Inside JSX, use curly braces to insert JavaScript values

You can also destructure props (extract them) to make code cleaner:

```
function Greeting({ name }) {  
  return <Text>Hello, {name}!</Text>;  
}
```

This is the same as above, but `name` is extracted directly instead of accessing `props.name`.

State: Data That Changes

State is data that can change over time. When state changes, React automatically updates the screen.

Think of state like a light switch:

- State: `isOn` (true or false)
- When state changes from `false` to `true`, the light turns on
- When state changes from `true` to `false`, the light turns off

In React, we use the `useState` hook:

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <View>
      <Text>{count}</Text>
      <Pressable onPress={() => setCount(count + 1)}>
        <Text>Increment</Text>
      </Pressable>
    </View>
  );
}
```

Breaking this down:

- `import { useState } from 'react'` — Import the `useState` function from React
- `const [count, setCount] = useState(0)` — Create a state variable:
 - `count` — The current value (starts at 0)
 - `setCount` — A function to change the value
 - `useState(0)` — Initialize the state to 0
- `{count}` — Display the current value
- `onPress={() => setCount(count + 1)}` — When pressed, add 1 to count

Why is this useful? Without state, when you click a button, nothing happens. With state, clicking a button changes the data, and React automatically updates the screen.

Hooks: Special Functions

Hooks are special functions that let you use React features. They always start with `use`:

- `useState` — Manage state (data that changes)
- `useEffect` — Run code when the component loads or when data changes
- `useCallback` — Memoize (remember) a function so it doesn't get recreated
- `useFocusEffect` — Run code when a screen comes into focus

We'll learn about these as we build the app.

Part 4: React Native Basics

React Native is a framework that lets you build mobile apps using React. Instead of writing separate code for iOS and Android, you write once and it works on both.

React Native Components

React Native has different components than web React:

React (Web)	React Native (Mobile)	What it does
<code><div></code>	<code><View></code>	Container
<code><p></code>	<code><Text></code>	Text
<code><input></code>	<code><TextInput></code>	Input field
<code><button></code>	<code><Pressable></code>	Clickable element
<code>/</code>	<code><FlatList></code>	List
<code></code>	<code><Image></code>	Image

Styling in React Native

In web React, you use CSS. In React Native, you use JavaScript objects:

```
// Web React (CSS):
.button {
  background-color: blue;
  padding: 10px;
  border-radius: 5px;
}

// React Native (JavaScript):
const buttonStyle = {
  backgroundColor: 'blue',
  padding: 10,
  borderRadius: 5
};

<Pressable style={buttonStyle}>
  <Text>Click Me</Text>
</Pressable>
```

Notice:

- CSS uses `background-color`, React Native uses `backgroundColor` (camelCase)
- CSS uses `px` (pixels), React Native just uses numbers
- You pass style as an object with `style={}`

Flexbox: Layout System

React Native uses **Flexbox** for layout. Flexbox automatically arranges items on the screen.

```
<View style={{ flex: 1, flexDirection: 'row', gap: 10 }}>
  <View style={{ flex: 1, backgroundColor: 'red' }} />
  <View style={{ flex: 1, backgroundColor: 'blue' }} />
  <View style={{ flex: 1, backgroundColor: 'green' }} />
</View>
```

Breaking this down:

- `flex: 1` — Take up available space
- `flexDirection: 'row'` — Arrange items horizontally (left to right)

- `'column'` would arrange vertically (top to bottom)
- `gap: 10` — Space between items (10 pixels)

Part 5: Understanding the File Structure

When you create a React Native app, you get many files. Let's understand what they do:

Key Files and Folders

```
mosher-lawns-time-tracker/
├── app/                                ← Your screens (pages)
│   ├── (tabs)/
│   │   ├── index.tsx                 ← Clock In/Out screen
│   │   └── logs.tsx                  ← Logs screen
│   ├── edit-log.tsx                  ← Edit entry screen
│   ├── employees-management.tsx      ← Manage employees screen
│   └── _layout.tsx                   ← Navigation setup
├── lib/
│   └── local-storage.ts              ← Data storage code
├── components/                       ← Reusable components
├── app.json                          ← App configuration
└── package.json                      ← Project dependencies
```

What is a .tsx File?

`.tsx` stands for “TypeScript JSX”. It’s a file that contains:

- TypeScript code (for type safety)
- JSX code (for UI)

What is package.json?

`package.json` is a file that lists all the libraries your project uses. Think of it like a shopping list:

```
{
  "dependencies": {
    "react": "19.1.0",
    "react-native": "0.81.5",
    "expo": "~54.0.29"
  }
}
```

This says: “This project needs React version 19.1.0, React Native version 0.81.5, and Expo version 54.”

What is export?

Export means “make this code available to other files”. Think of it like publishing a book:

```
// In file: lib/local-storage.ts
export function addEmployee(name: string) {
  // Code here
}

// In file: app/employees-management.tsx
import { addEmployee } from '@lib/local-storage';

// Now we can use addEmployee in this file
addEmployee("John");
```

Breaking this down:

- `export function addEmployee()` — “I’m making this function available to other files”
- `import { addEmployee }` — “I want to use the addEmployee function from another file”
- `from '@lib/local-storage'` — “It’s in the local-storage file”

Why use export/import? Because you want to organize code into separate files. You might have 100 lines of data storage code, and you don’t want to put it all in one screen file. So you put it in `local-storage.ts` and import it where you need it.

Part 6: Building the Data Storage Layer

Now let's build the code that stores data. This is where we'll use interfaces and export/import.

Understanding AsyncStorage

AsyncStorage is a way to save data on the phone that persists (stays there) even after the app closes.

Think of it like:

- **Memory (State):** A whiteboard. You can write on it, but when you erase it, it's gone.
- **AsyncStorage:** A notebook. You can write in it, and even if you close the notebook, the writing is still there next time you open it.

Creating the Storage File

Create a new file: `lib/local-storage.ts`

```

import AsyncStorage from '@react-native-async-storage/async-storage';

// =====
// INTERFACES (Data Blueprints)
// =====

/**
 * An Employee is a person who works on jobs.
 *
 * Example:
 * {
 *   id: "emp-001",
 *   name: "John Smith"
 * }
 */
export interface Employee {
  id: string;           // Unique identifier
  name: string;         // Person's name
}

/**
 * A JobSite is a location where work happens.
 *
 * Example:
 * {
 *   id: "job-001",
 *   name: "Downtown Site",
 *   location: "123 Main St"
 * }
 */
export interface JobSite {
  id: string;           // Unique identifier
  name: string;         // Job site name
  location: string;     // Address
}

/**
 * A TimeLog is a record of when an employee worked.
 *
 * Example:
 * {
 *   id: "log-001",
 *   employeeId: "emp-001",

```

```

*   jobSiteId: "job-001",
*   clockInTime: 1717689600000,
*   clockOutTime: 1717707600000
* }
*/
export interface TimeLog {
  id: string;           // Unique identifier
  employeeId: string;   // Which employee (references Employee.id)
  jobSiteId: string;    // Which job site (references JobSite.id)
  clockInTime: number;  // When they clocked in (milliseconds)
  clockOutTime: number | null; // When they clocked out (null if still working)
}

// =====
// STORAGE KEYS
// =====

/**
 * These are the names we use to store data in AsyncStorage.
 * Think of them like file names on a computer.
 */
const EMPLOYEES_KEY = 'employees';
const JOB_SITES_KEY = 'jobSites';
const TIME_LOGS_KEY = 'timeLogs';

// =====
// EMPLOYEE OPERATIONS
// =====

/**
 * Get all employees from storage.
 *
 * Returns: An array of employees
 * Example: [{ id: "emp-001", name: "John" }, { id: "emp-002", name: "Jane" }]
 */
export async function getEmployees(): Promise<Employee[]> {
  // Get the data from AsyncStorage
  const data = await AsyncStorage.getItem(EMPLOYEES_KEY);

  // If no data exists, return empty array
  if (!data) {
    return [];
  }
}

```

```

    // Convert the text data back to JavaScript objects
    return JSON.parse(data);
}

/**
 * Add a new employee.
 *
 * Steps:
 * 1. Get all existing employees
 * 2. Create a new employee object
 * 3. Add it to the array
 * 4. Save the array back to storage
 */
export async function addEmployee(name: string): Promise<void> {
    // Step 1: Get existing employees
    const employees = await getEmployees();

    // Step 2: Create new employee
    const newEmployee: Employee = {
        id: generateUniqueId(), // Create a unique ID
        name: name,
    };

    // Step 3: Add to array
    employees.push(newEmployee);

    // Step 4: Save back to storage
    await AsyncStorage.setItem(EMPLOYEES_KEY, JSON.stringify(employees));
}

/**
 * Delete an employee by ID.
 */
export async function deleteEmployee(id: string): Promise<void> {
    // Get all employees
    const employees = await getEmployees();

    // Keep only employees that don't have this ID
    const filtered = employees.filter(e => e.id !== id);

    // Save the filtered list
    await AsyncStorage.setItem(EMPLOYEES_KEY, JSON.stringify(filtered));
}

```



```

// =====
// JOB SITE OPERATIONS
// =====

/**
 * Get all job sites from storage.
 */
export async function getJobSites(): Promise<JobSite[]> {
  const data = await AsyncStorage.getItem(JOB_SITES_KEY);
  return data ? JSON.parse(data) : [];
}

/**
 * Add a new job site.
 */
export async function addJobSite(name: string, location: string): Promise<void> {
  const jobs = await getJobSites();

  const newJob: JobSite = {
    id: generateUniqueId(),
    name: name,
    location: location,
  };

  jobs.push(newJob);
  await AsyncStorage.setItem(JOB_SITES_KEY, JSON.stringify(jobs));
}

/**
 * Delete a job site by ID.
 */
export async function deleteJobSite(id: string): Promise<void> {
  const jobs = await getJobSites();
  const filtered = jobs.filter(j => j.id !== id);
  await AsyncStorage.setItem(JOB_SITES_KEY, JSON.stringify(filtered));
}

// =====
// TIME LOG OPERATIONS
// =====

/**
 * Get all time logs from storage.
 */

```

```

export async function getTimeLogs(): Promise<TimeLog[]> {
  const data = await AsyncStorage.getItem(TIME_LOGS_KEY);
  return data ? JSON.parse(data) : [];
}

/**
 * Create a new time log (clock in).
 *
 * This creates a log with:
 * - Current time as clock in time
 * - null as clock out time (not clocked out yet)
 */
export async function addTimeLog(
  employeeId: string,
  jobSiteId: string
): Promise<void> {
  const logs = await getTimeLogs();

  const newLog: TimeLog = {
    id: generateUniqueId(),
    employeeId: employeeId,
    jobSiteId: jobSiteId,
    clockInTime: Date.now(), // Current time in milliseconds
    clockOutTime: null,      // Not clocked out yet
  };

  logs.push(newLog);
  await AsyncStorage.setItem(TIME_LOGS_KEY, JSON.stringify(logs));
}

/**
 * Clock out (update a time log with clock out time).
 */
export async function clockOut(logId: string): Promise<void> {
  const logs = await getTimeLogs();

  // Find the log with this ID
  const log = logs.find(l => l.id === logId);

  if (log) {
    // Update its clock out time
    log.clockOutTime = Date.now();

    // Save back to storage
  }
}

```

```

    await AsyncStorage.setItem(TIME_LOGS_KEY, JSON.stringify(logs));
  }
}

/**
 * Delete a time log.
 */
export async function deleteTimeLog(id: string): Promise<void> {
  const logs = await getTimeLogs();
  const filtered = logs.filter(l => l.id !== id);
  await AsyncStorage.setItem(TIME_LOGS_KEY, JSON.stringify(filtered));
}

/**
 * Manually add a time log with specific times.
 * Used when entering past time entries.
 */
export async function addManualTimeLog(
  employeeId: string,
  jobSiteId: string,
  clockInTime: number,
  clockOutTime: number | null
): Promise<void> {
  const logs = await getTimeLogs();

  const newLog: TimeLog = {
    id: generateUniqueId(),
    employeeId: employeeId,
    jobSiteId: jobSiteId,
    clockInTime: clockInTime,
    clockOutTime: clockOutTime,
  };

  logs.push(newLog);
  await AsyncStorage.setItem(TIME_LOGS_KEY, JSON.stringify(logs));
}

// =====
// UTILITY FUNCTIONS
// =====

/**
 * Generate a unique ID.
 * This creates a random string that's different every time.

```

```

    */
function generateUniqueId(): string {
    return Math.random().toString(36).substring(2, 11);
}

/**
 * Get all time logs for a specific employee.
 */
export async function getEmployeeTimeLogs(employeeId: string): Promise<TimeLog[]> {
    const logs = await getTimeLogs();
    return logs.filter(log => log.employeeId === employeeId);
}

/**
 * Get all time logs for a specific job site.
 */
export async function getJobSiteTimeLogs(jobSiteId: string): Promise<TimeLog[]> {
    const logs = await getTimeLogs();
    return logs.filter(log => log.jobSiteId === jobSiteId);
}

/**
 * Get currently active time logs (people who are clocked in).
 */
export async function getActiveLogs(): Promise<TimeLog[]> {
    const logs = await getTimeLogs();
    return logs.filter(log => log.clockOutTime === null);
}

/**
 * Calculate total hours worked by an employee.
 */
export async function getEmployeeTotalHours(employeeId: string): Promise<number> {
    const logs = await getEmployeeTimeLogs(employeeId);
    let totalMilliseconds = 0;

    logs.forEach(log => {
        if (log.clockOutTime) {
            // Calculate milliseconds worked
            totalMilliseconds += log.clockOutTime - log.clockInTime;
        }
    });

    // Convert milliseconds to hours

```

```

    return totalMilliseconds / 3600000;
}

/**
 * Export all logs as CSV for reporting.
 * CSV is a format that can be opened in Excel.
 */
export async function exportLogsToCSV(): Promise<string> {
    const logs = await getTimeLogs();
    const employees = await getEmployees();
    const jobs = await getJobSites();

    // Create lookup maps for fast access
    const empMap = new Map(employees.map(e => [e.id, e.name]));
    const jobMap = new Map(jobs.map(j => [j.id, j.name]));

    // Start with CSV header (column names)
    let csv = 'Employee,Job Site,Clock In,Clock Out,Duration (hours)\n';

    // Add each log as a row
    logs.forEach(log => {
        const empName = empMap.get(log.employeeId) || 'Unknown';
        const jobName = jobMap.get(log.jobSiteId) || 'Unknown';
        const clockIn = new Date(log.clockInTime).toISOString();
        const clockOut = log.clockOutTime
            ? new Date(log.clockOutTime).toISOString()
            : 'In Progress';
        const duration = log.clockOutTime
            ? ((log.clockOutTime - log.clockInTime) / 3600000).toFixed(2)
            : 'N/A';

        csv += `${empName},${jobName},${clockIn},${clockOut},${duration}\n`;
    });

    return csv;
}

```

Understanding the Code

Let's break down what we just wrote:

Interfaces:

```
export interface Employee {  
  id: string;  
  name: string;  
}
```

This says: “An Employee must have an `id` (text) and a `name` (text). Nothing more, nothing less.”

Export:

```
export function getEmployees(): Promise<Employee[]> {  
  // ...  
}
```

The `export` keyword makes this function available to other files. Other files can import and use it.

Promise:

```
export async function getEmployees(): Promise<Employee[]> {
```

`Promise<Employee[]>` means: “This function will eventually return an array of employees, but it might take time.” The `async` keyword means the function is asynchronous (it takes time).

JSON.stringify and JSON.parse:

```
// Convert object to text  
JSON.stringify(employees) // [{"id":"emp-001","name":"John"}]  
  
// Convert text back to object  
JSON.parse(data)          // [{ id: "emp-001", name: "John" }]
```

AsyncStorage only stores text, so we convert objects to text before saving and back to objects after loading.

Part 7: Building the Clock In/Out Screen

Now let's build the first screen. Create `app/(tabs)/index.tsx` :

```

import { useState, useEffect, useCallback } from 'react';
import { View, Text, ScrollView, Pressable, FlatList } from 'react-native';
import { useFocusEffect } from 'expo-router';
import * as Storage from '@lib/local-storage';
import { useColors } from '@hooks/use-colors';

/**
 * This is the main screen where employees clock in and out.
 *
 * What it does:
 * 1. Shows a list of employees
 * 2. Shows a list of job sites
 * 3. Lets you select an employee and job
 * 4. Has a "Clock In" button
 * 5. Shows who is currently clocked in
 * 6. Lets you clock out
 */
export default function ClockInOutScreen() {
  const colors = useColors();

  // =====
  // STATE VARIABLES
  // =====

  /**
   * employees: List of all employees
   * setEmployees: Function to update the list
   */
  const [employees, setEmployees] = useState<Storage.Employee[]>([]);

  /**
   * jobs: List of all job sites
   * setJobs: Function to update the list
   */
  const [jobs, setJobs] = useState<Storage.JobSite[]>([]);

  /**
   * selectedEmployee: The ID of the currently selected employee
   * setSelectedEmployee: Function to change which employee is selected
   */
  const [selectedEmployee, setSelectedEmployee] = useState<string>('');

  /**

```



```

    * selectedJob: The ID of the currently selected job
    * setSelectedJob: Function to change which job is selected
    */
const [selectedJob, setSelectedJob] = useState<string>('');

/**
 * activeLogs: List of time logs for people currently clocked in
 * setActiveLogs: Function to update the list
 */
const [activeLogs, setActiveLogs] = useState<Storage.TimeLog[]>([]);

// =====
// LOAD DATA WHEN SCREEN COMES INTO FOCUS
// =====

/**
 * useEffect runs code when this screen comes into focus.
 * This means every time you navigate to this screen, it loads fresh data
 */
useEffect(
  useCallback(() => {
    loadData();
  }, [])
);

// =====
// LOAD DATA FROM STORAGE
// =====

const loadData = async () => {
  // Fetch all data from storage
  const emps = await Storage.getEmployees();
  const jbs = await Storage.getJobSites();
  const logs = await Storage.getTimeLogs();

  // Update state with fetched data
  setEmployees(emps);
  setJobs(jbs);

  // Filter to show only active logs (people still clocked in)
  // A log is active if clockOutTime is null
  setActiveLogs(logs.filter(l => !l.clockOutTime));
};

```

```

// =====
// HANDLE CLOCK IN
// =====

const handleClockIn = async () => {
  // Check that both employee and job are selected
  if (!selectedEmployee || !selectedJob) {
    alert('Please select an employee and job site');
    return;
  }

  // Create new time log in storage
  await Storage.addTimeLog(selectedEmployee, selectedJob);

  // Reload data to show new log
  loadData();
};

// =====
// HANDLE CLOCK OUT
// =====

const handleClockOut = async (logId: string) => {
  // Update the time log with clock out time
  await Storage.clockOut(logId);

  // Reload data to update UI
  loadData();
};

// =====
// UTILITY FUNCTIONS
// =====

/**
 * Convert a timestamp (milliseconds) to a readable time string.
 * Example: 1717689600000 becomes "8:00:00 AM"
 */
const formatTime = (timestamp: number) => {
  return new Date(timestamp).toLocaleTimeString();
};

// =====
// RENDER (What appears on screen)

```

```
// =====

return (
  <ScrollView style={{ flex: 1, backgroundColor: colors.background }}>
    <View style={{ padding: 16, gap: 24 }}>

      {/* Header */}
      <View>
        <Text style={{ fontSize: 28, fontWeight: 'bold', color: colors.primary }}>
          Clock In/Out
        </Text>
        <Text style={{ fontSize: 14, color: colors.muted }}>
          Select employees and job site
        </Text>
      </View>

      {/* Employee Selection */}
      <View style={{ gap: 8 }}>
        <Text style={{ fontSize: 14, fontWeight: 'bold', color: colors.primary }}>
          Select Employee
        </Text>

        {/* Show each employee as a button */}
        <FlatList
          data={employees}
          keyExtractor={e => e.id}
          scrollEnabled={false}
          renderItem={({ item }) => (
            <Pressable
              onPress={() => setSelectedEmployee(item.id)}
              style={{
                backgroundColor: selectedEmployee === item.id ? colors.primary : colors.background,
                padding: 12,
                marginBottom: 8,
                borderRadius: 8,
                borderWidth: 1,
                borderColor: colors.border,
              }}
            >
              <Text
                style={{
                  color: selectedEmployee === item.id ? 'white' : colors.primary,
                  fontWeight: 'bold',
                }}
              />
            </Pressable>
          )}
        />
      </View>
    </View>
  </ScrollView>
)
```

```

        >
        {item.name}
      </Text>
    </Pressable>
  })
  />
</View>

{/* Job Site Selection */}
<View style={{ gap: 8 }}>
  <Text style={{ fontSize: 14, fontWeight: '600', color: colors.foreground }}>
    Select Job Site
  </Text>

  {/* Show each job as a button */}
  <FlatList
    data={jobs}
    keyExtractor={j => j.id}
    scrollEnabled={false}
    renderItem={({ item }) => (
      <Pressable
        onPress={() => setSelectedJob(item.id)}
        style={{
          backgroundColor: selectedJob === item.id ? colors.primary : colors.background,
          padding: 12,
          marginBottom: 8,
          borderRadius: 8,
          borderWidth: 1,
          borderColor: colors.border,
        }}
      >
        <Text
          style={{
            color: selectedJob === item.id ? 'white' : colors.foreground,
            fontWeight: '500',
          }}
        >
          {item.name}
        </Text>
        <Text style={{ fontSize: 12, color: colors.muted }}>
          {item.location}
        </Text>
      </Pressable>
    )}
  >

```

```

    />
  </View>

  {/* Clock In Button */}
  <Pressable
    onPress={handleClockIn}
    style={{
      backgroundColor: colors.success,
      padding: 16,
      borderRadius: 8,
      alignItems: 'center',
    }}
  >
    <Text style={{ color: 'white', fontWeight: '600', fontSize: 16 }}>
      Clock In
    </Text>
  </Pressable>

  {/* Currently Clock In Section */}
  <View style={{ gap: 12 }}>
    <Text style={{ fontSize: 16, fontWeight: '600', color: colors.f }}>
      Currently Clock In
    </Text>

    {activeLogs.length === 0 ? (
      <Text style={{ color: colors.muted }}>No active time entries<
    ) : (
      <FlatList
        data={activeLogs}
        keyExtractor={l => l.id}
        scrollEnabled={false}
        renderItem={({ item }) => {
          // Look up employee and job names by ID
          const emp = employees.find(e => e.id === item.employeeId)
          const job = jobs.find(j => j.id === item.jobSiteId);

          return (
            <View
              style={{
                backgroundColor: colors.surface,
                padding: 12,
                borderRadius: 8,
                borderLeftWidth: 4,
                borderLeftColor: colors.success,

```

```

        gap: 8,
      }}
    >
    <Text style={{ fontWeight: '600', color: colors.foreground }}>
      {emp?.name}
    </Text>
    <Text style={{ fontSize: 12, color: colors.muted }}>
      {job?.name}
    </Text>
    <Text style={{ fontSize: 12, color: colors.muted }}>
      Clocked in at {formatTime(item.clockInTime)}
    </Text>
    <Pressable
      onPress={() => handleClockOut(item.id)}
      style={{
        backgroundColor: colors.error,
        padding: 10,
        borderRadius: 6,
        alignItems: 'center',
      }}
    >
      <Text style={{ color: 'white', fontWeight: '600' }}>
        Clock Out
      </Text>
    </Pressable>
  </View>
);
}}
/>
)}
</View>
</View>
</ScrollView>
);
}

```

Understanding This Screen

State Variables:

```
const [employees, setEmployees] = useState<Storage.Employee[]>([]);
```

This creates a state variable:

- `employees` — The current list of employees
- `setEmployees` — Function to update the list
- `useState<Storage.Employee[]>([])` — Initialize as empty array, with type `Storage.Employee[]` (array of Employee objects)

useFocusEffect:

```
useFocusEffect(  
  useCallback(() => {  
    loadData();  
  }, [])  
);
```

This runs `loadData()` every time this screen comes into focus. It ensures we always have fresh data.

FlatList:

```
<FlatList  
  data={employees}  
  keyExtractor={e => e.id}  
  renderItem={({ item }) => <Pressable>...</Pressable>}  
</>
```

This displays a list:

- `data={employees}` — The list to display
- `keyExtractor={e => e.id}` — Use the ID as the unique key for each item
- `renderItem={({ item }) => ...}` — For each item, render this component

Conditional Rendering:

```
{activeLogs.length === 0 ? (  
  <Text>No active time entries</Text>  
) : (  
  <FlatList>...</FlatList>  
)}
```

The `?` and `:` is a ternary operator. It means:

- If `activeLogs.length === 0` is true, show “No active time entries”
- Otherwise (`:`) show the FlatList

Part 8: Building the Logs Screen

Create `app/(tabs)/logs.tsx`:


```

import { useEffect, useState, useCallback } from 'react';
import { View, Text, ScrollView, FlatList, Pressable, Alert } from 'react-native';
import { useFocusEffect } from 'expo-router';
import * as Storage from '@lib/local-storage';
import { useColors } from '@hooks/use-colors';

/**
 * This screen shows all time logs.
 * It groups logs by job site and time to reduce clutter.
 */

// =====
// INTERFACE FOR GROUPED LOGS
// =====

/**
 * LogGroup represents multiple logs that should be displayed together.
 *
 * Example:
 * {
 *   logs: [
 *     { id: "log-001", employeeId: "emp-001", jobSiteId: "job-001", ... },
 *     { id: "log-002", employeeId: "emp-002", jobSiteId: "job-001", ... },
 *   ],
 *   firstLog: { id: "log-001", ... }
 * }
 *
 * This represents 2 employees working the same job at the same time.
 */
interface LogGroup {
  logs: Storage.TimeLog[];
  firstLog: Storage.TimeLog;
}

export default function LogsScreen() {
  const colors = useColors();

  // =====
  // STATE VARIABLES
  // =====

  const [logs, setLogs] = useState<Storage.TimeLog[]>([]);
  const [employees, setEmployees] = useState<Map<string, string>>(new Map

```

```

const [jobs, setJobs] = useState<Map<string, Storage.JobSite>>(new Map());

useFocusEffect(
  useCallback(() => {
    loadData();
  }, [])
);

// =====
// LOAD DATA FROM STORAGE
// =====

const loadData = async () => {
  const allLogs = await Storage.getTimeLogs();
  const emps = await Storage.getEmployees();
  const jbs = await Storage.getJobSites();

  // Create Maps for fast lookup by ID
  // Map is like a dictionary: { key: value }
  // empMap: { "emp-001": "John Smith", "emp-002": "Jane Doe" }
  const empMap = new Map(emps.map(e => [e.id, e.name]));
  const jobMap = new Map(jbs.map(j => [j.id, j]));

  // Sort logs by most recent first
  setLogs(allLogs.sort((a, b) => b.clockInTime - a.clockInTime));
  setEmployees(empMap);
  setJobs(jobMap);
};

// =====
// GROUP LOGS BY JOB AND TIME WINDOW
// =====

/**
 * This function groups logs together if they have the same:
 * - Job site ID
 * - Clock in time
 * - Clock out time
 *
 * This reduces clutter when multiple employees work the same job at the
 */
const groupLogs = (): LogGroup[] => {
  // Create an object to hold groups
  // groupKey: { logs: [...], firstLog: ... }

```

```

const grouped: Record<string, LogGroup> = {};

logs.forEach(log => {
  // Create a unique key for this group
  // Example: "job-001-1717689600000-1717707600000"
  const key = `${log.jobSiteId}-${log.clockInTime}-${log.clockOutTime}`

  // If this group doesn't exist yet, create it
  if (!grouped[key]) {
    grouped[key] = { logs: [], firstLog: log };
  }

  // Add this log to the group
  grouped[key].logs.push(log);
});

// Convert object to array
// Object.values() gets all the values from an object
return Object.values(grouped);
};

// =====
// HANDLE DELETE LOG
// =====

const handleDeleteLog = async (id: string) => {
  Alert.alert('Delete?', 'Are you sure?', [
    { text: 'Cancel' },
    {
      text: 'Delete',
      onPress: async () => {
        await Storage.deleteTimeLog(id);
        loadData();
      },
    },
  ]);
};

// =====
// UTILITY FUNCTIONS
// =====

const formatTime = (timestamp: number) => {
  return new Date(timestamp).toLocaleTimeString();
};

```

```

};

const calculateDuration = (clockIn: number, clockOut: number | null) =>
  if (!clockOut) return 'In Progress';

  // Calculate difference in milliseconds
  const milliseconds = clockOut - clockIn;

  // Convert to minutes
  const minutes = Math.floor(milliseconds / 60000);

  // Convert to hours and remaining minutes
  const hours = Math.floor(minutes / 60);
  const mins = minutes % 60;

  return `${hours}h ${mins}m`;
};

// =====
// RENDER
// =====

const groupedLogs = groupLogs();

return (
  <ScrollView style={{ flex: 1, backgroundColor: colors.background }}>
    <View style={{ padding: 16, gap: 24 }}>
      <View>
        <Text style={{ fontSize: 28, fontWeight: 'bold', color: colors.muted }}>
          Time Logs
        </Text>
        <Text style={{ fontSize: 14, color: colors.muted }}>
          View and manage time entries
        </Text>
      </View>

      {logs.length === 0 ? (
        <Text style={{ color: colors.muted, textAlign: 'center' }}>
          No time entries yet
        </Text>
      ) : (
        <FlatList
          data={groupedLogs}
          keyExtractor={({_, idx}) => idx.toString()}
        >

```

```

scrollEnabled={false}
renderItem={({ item: group }) => {
  const firstLog = group.firstLog;

  // Get employee names for all logs in this group
  const empNames = group.logs
    .map(log => employees.get(log.employeeId) || 'Unknown')
    .join(', ');

  // Get job site info
  const jobSite = jobs.get(firstLog.jobSiteId);

  // Calculate duration
  const duration = calculateDuration(firstLog.clockInTime, firstLog.clockOutTime);

  // Check if still active (not clocked out)
  const isActive = !firstLog.clockOutTime;

  return (
    <View
      style={{
        backgroundColor: colors.surface,
        borderRadius: 12,
        padding: 16,
        marginBottom: 12,
        borderLeftWidth: 4,
        borderLeftColor: isActive ? colors.success : colors.error,
      }}
    >
      {/* Employee Names */}
      <Text style={{ fontWeight: '600', color: colors.foreground }}>
        {empNames}
      </Text>

      {/* Badge showing number of employees */}
      {group.logs.length > 1 && (
        <View
          style={{
            backgroundColor: colors.primary,
            borderRadius: 12,
            paddingVertical: 4,
            paddingHorizontal: 8,
            alignSelf: 'flex-start',
            marginTop: 4,
          }}
        >
          {group.logs.length}
        </View>
      )}
    </View>
  );
}

```

```

    }}
  >
    <Text style={{ fontSize: 11, fontWeight: '600', color: colors.muted }}>
      {group.logs.length} employees
    </Text>
  </View>
)}

{/* Job Site Name */}
<Text style={{ fontSize: 12, color: colors.muted, margin: 0 10 }}>
  {jobSite?.name}
</Text>

{/* Time Details */}
<View
  style={{
    backgroundColor: colors.background,
    borderRadius: 8,
    padding: 12,
    marginTop: 12,
    gap: 8,
  }}
>
  <View style={{ flexDirection: 'row', justifyContent: 'space-between' }}>
    <Text style={{ fontSize: 12, color: colors.muted }}>
      {formatTime(firstLog.clockInTime)}
    </Text>
  </View>
  {firstLog.clockOutTime && (
    <View style={{ flexDirection: 'row', justifyContent: 'space-between' }}>
      <Text style={{ fontSize: 12, color: colors.muted }}>
        {formatTime(firstLog.clockOutTime)}
      </Text>
    </View>
  )}
  <View
    style={{
      flexDirection: 'row',
      justifyContent: 'space-between',
      padding: 8,
      border: 1px solid colors.border,
    }}
  >
    <Text style={{ fontSize: 12, color: colors.muted }}>
      {formatTime(firstLog.clockInTime)}
    </Text>
    <Text style={{ fontSize: 12, color: colors.muted }}>
      {formatTime(firstLog.clockOutTime)}
    </Text>
  </View>

```

```

    }}
  >
    <Text style={{ fontSize: 12, fontWeight: '600', color: 'white',
      Duration
    </Text>
    <Text style={{ fontSize: 12, fontWeight: '600', color: 'white',
      {duration}
    </Text>
  </View>
</View>

{/* Action Buttons */}
<View style={{ flexDirection: 'row', gap: 8, marginTop: 10,
  <Pressable
    onPress={() => handleDeleteLog(firstLog.id)}
    style={{
      flex: 1,
      backgroundColor: colors.error,
      padding: 10,
      borderRadius: 8,
      alignItems: 'center',
    }}
  >
    <Text style={{ color: 'white', fontWeight: '600', fontSize: 12,
      Delete
    </Text>
  </Pressable>
</View>
</View>
);
}}
/>
})
</View>
</ScrollView>
);
}

```

Part 9: Key Concepts Summary

What We've Learned

JavaScript:

- Variables store information
- Data types (string, number, boolean, null)
- Operators (math, comparison, logical)
- Functions are reusable instructions
- Objects group related data
- Arrays are lists
- Loops repeat instructions
- Conditionals make decisions

TypeScript:

- Types add safety
- Interfaces describe object structure
- Union types allow multiple types

React:

- Components are reusable UI pieces
- JSX is HTML-like syntax in JavaScript
- Props pass data to components
- State is data that changes
- Hooks are special functions (useState, useEffect, etc.)

React Native:

- Components are different from web (View, Text, Pressable, etc.)
- Styling uses JavaScript objects
- Flexbox arranges items on screen

Our App:

- Data storage with AsyncStorage
 - CRUD operations (Create, Read, Update, Delete)
 - Relationships between data (employees, jobs, logs)
 - UI screens that display and manage data
-

Conclusion

You've now learned the fundamentals of programming and built a complete mobile app! You understand:

- What programming languages are and why we use them
- JavaScript basics (variables, functions, objects, arrays)
- TypeScript and why it's useful
- React and how components work
- React Native and how to build mobile UIs
- How to store data and manage it
- How to build screens that interact with data

This knowledge is the foundation for all mobile app development. The concepts you've learned here apply to any programming language and any framework.

Next Steps

From here, you can:

- Add more screens (settings, reports, etc.)
- Add more features (notifications, camera, location)
- Connect to a backend server for cloud sync
- Learn about performance optimization
- Publish your app to the App Store and Google Play

Happy coding!